

Aula 10 – Tratamento de Exceções

Programação Orientada a Objetos com C#
Professor: Eduardo Luan Pilonetto

Objetivos da Aula

- • Tratar erros adequadamente
- • Evitar falhas no sistema
- • Melhorar robustez
- • Aplicar boas práticas em C#

Introdução ao Tratamento de Exceções

- • O que são exceções
- • Diferença entre erro de compilação e execução
- • Impacto das falhas no sistema

Estrutura try/catch

- Bloco try
- Bloco catch
- Captura de erros específicos
- Fluxo de execução

Exemplo Básico try/catch

```
try {  
    int x = 10 / 0;  
}  
catch (DivideByZeroException ex) {  
    Console.WriteLine(ex.Message);  
}
```

Múltiplos catch

- Captura de diferentes exceções
- Ordem correta dos blocos
- Exceções mais específicas primeiro

Bloco finally

- • Sempre executado
- • Liberação de recursos
- • Fechamento de conexões e arquivos

Exemplo com finally

```
try {  
    Console.WriteLine("Abrindo arquivo");  
}  
finally {  
    Console.WriteLine("Fechando arquivo");  
}
```

Throw e Relançamento

- • Uso da palavra-chave throw
- • Relançar exceções
- • Encadeamento de exceções

Exceções Personalizadas

- Criando classes de exceção
- Herança de Exception
- Mensagens personalizadas

Exemplo de Exceção Personalizada

```
public class SaldoInsuficienteException : Exception {  
    public SaldoInsuficienteException(string msg) : base(msg) {}  
}
```

Boas Práticas

- • Não capturar Exception sem necessidade
- • Evitar catch vazio
- • Registrar logs
- • Mensagens claras

Validação Preventiva

- Validar entradas
- Uso de TryParse
- Evitar exceções desnecessárias

Tratamento em Camadas

- • Separação de responsabilidades
- • Camada de negócio
- • Camada de apresentação

Logs e Monitoramento

- • Registrar falhas
- • Ferramentas de logging
- • Diagnóstico de problemas

Resumo da Aula

- • try/catch/finally
- • Exceções personalizadas
- • Robustez
- • Boas práticas

Encerramento

- Dúvidas?
- Próxima aula: UML e Modelagem